Formularios reactivos: alta de productos de inventario

App `inventario-app` · Ionic 9 + Angular 22 standalone · Reactive Forms + Signals + Capacitor Preferences
Jorge González Gómez · 2º DAM · Desarrollo de Interfaces (0488) · Universidad Nebrija · versión 2, 26-10-2026
Código: github.com/JorgeGonzalezDI/Desarrollo-Interfaces-DAM-2/tree/main/07-Formularios-Reactivos-Inventario

Práctica completa. Hay un formulario reactivo con 4 campos, 3 validadores propios y mensajes de error. Un signal bloquea el formulario mientras guarda, sale un toast y los datos se guardan en el dispositivo. Todo el código está comentado en GitHub.

App general de repaso. Para entender bien los conceptos de las prácticas 04–08 sin rehacerlas una a una, estoy haciendo una sola app de gestión con roles que junta lo importante de cada una (Supabase, guards, formularios reactivos y estado con signals). Está en el mismo repositorio: rama `app-gestion-roles`, carpeta `09-App-Gestion-Roles`.

1. Estructura del proyecto

Separé el modelo, los validadores y el guardado en su propio fichero, para que la página solo se encargue de la vista:

```
inventario-app/src/app/
|-- models/producto.model.ts          CATEGORIAS, tipo Categoria e interfaces Producto
|-- validators/producto.validators.ts 3 validadores personalizados (+ .spec.ts con sus tests)
|-- services/inventario.service.ts    "petición" simulada + guardado local (Preferences)
`-- pages/nuevo-producto/             la vista: .ts (lógica) + .html (plantilla) + .scss
```

2. El formulario reactivo

Todo el formulario se define en TypeScript con `FormBuilder`. En el HTML solo se enlaza con `[formGroup]` y `formControlName`:

```
protected readonly form = this.fb.nonNullable.group({
  sku:         ['',   [Validators.required, skuValidator(12)]],
  nombre:      ['',   [Validators.required, noSoloEspaciosValidator, Validators.maxLength(60)]],
  categoria:   ['' as Categoria | '', Validators.required],
  stockMinimo: [null as number | null, [Validators.required, enteroPositivoValidator]],
});
```

Campo	Componente Ionic	Validación
Código de barras (SKU)	`<ion-input>` con `maxlength=12` y contador 0 / 12	Obligatorio + `skuValidator(12)`: solo letras o números y exactamente 12
Nombre	`<ion-input>`	Obligatorio + `noSoloEspaciosValidator` (rechaza « ») + máx. 60
Categoría	`<ion-select interface="action-sheet">`: Electrónica, Herramientas, Consumibles	Obligatorio
Stock mínimo	`<ion-input type="number" inputmode="numeric">` (teclado numérico)	Obligatorio + `enteroPositivoValidator`: mayor que 0 y entero

3. Validadores personalizados y mensajes de error

Un validador es una función que recibe el control y devuelve `null` si está bien, o un objeto con el error. Si el campo está vacío devuelven `null` y lo deja a `Validators.required`, así cada uno comprueba una sola cosa. `skuValidator` es una fábrica, igual que `Validators.minLength(3)`: recibe la longitud y devuelve el validador. Además devuelve datos en el error para construir el mensaje:

```
if (!/^[A-Za-z0-9]+$/.test(valor)) return { skuCaracteres: true };
if (valor.length !== longitud)    return { skuLongitud: { requerida: longitud, actual: valor.length } };
```

`mensajeError(campo)` mira qué error tiene cada control y devuelve el texto que se pasa a `[errorText]`. Ionic solo lo enseña cuando el campo es `ion-invalid` e `ion-touched`. **Cambio respecto al dossier:** allí el botón se desactiva con `registroForm.invalid`, así que nunca se llega a ejecutar `markAllAsTouched()`. Aquí el botón solo se bloquea mientras se guarda. Si el formulario es inválido, al pulsar se marcan todos los campos, se ven todos los errores a la vez y sale un toast rojo.

4. Estado del guardado con un `Signal<boolean>`

Un solo signal, `guardando`, controla todo lo que cambia mientras se guarda. Un `effect()` lo sincroniza con el formulario:

```
protected readonly guardando = signal<boolean>(false);

effect(() => {                              // en el constructor: el signal manda sobre el formulario
  if (this.guardando()) this.form.disable();
  else this.form.enable();
});

const datos = this.form.getRawValue();      // antes de desactivar: un form desactivado no da su .value
this.guardando.set(true);                   // -> form desactivado + spinner en el botón
try {
  const producto = await this.inventario.guardar({ ...datos, sku: datos.sku.toUpperCase(), ... });
  this.form.reset();
  this.abrirToast(`"${producto.nombre}" guardado en el dispositivo.`, 'success');
} catch { this.abrirToast('No se pudo guardar el producto...', 'danger'); }
finally { this.guardando.set(false); }      // se reactiva pase lo que pase
```

El toast es un `<ion-toast>` controlado también con signals: `[isOpen]="toastAbierto()"`, `[message]`, `[color]` y `(didDismiss)`.

5. Guardado local (offline first)

Como en el punto 1.4 del dossier, el destino es el propio dispositivo. `InventarioService` espera 1,5 s (simula la petición), añade `id` y `fechaAlta`, y guarda la lista como JSON con Capacitor `Preferences`. En Android o iOS va al almacenamiento nativo y en `ionic serve` a `localStorage`. La lista se expone en solo lectura con `asReadonly()`, como el Store de la práctica 05.

6. Comprobaciones

Qué	Resultado
Enviar vacío	`markAllAsTouched()` enseña los 4 errores y sale el toast rojo
Validadores propios	SKU de 6 → «llevas 6»; nombre solo con espacios → error; stock 2.5 → no entero; stock 0 → «mayor que 0»
Guardar válido	Campos desactivados y spinner; después toast verde, `reset()` y el producto aparece en la lista (SKU en mayúsculas)
Recargar (F5)	Los productos siguen ahí: están en el almacenamiento del dispositivo
`ng build` · `ng lint` · `ng test`	Sin errores · all files pass · 10 tests OK (9 de los validadores, probados sin DOM)

7. Ejecutar

En `07-Formularios-Reactivos-Inventario/inventario-app`: `npm install` y `ionic serve` (abre `http://localhost:8100/nuevo-producto`).